

Respostas - Lista de 20 questões

Resposta 1 - Responsabilidade Única

O princípio violado é o **Princípio da Responsabilidade Única**.

A classe `Pedido` possui mais de uma responsabilidade:

```
calcular o total do pedido;  
salvar no banco de dados;  
enviar e-mail de confirmação.
```

Essas responsabilidades podem mudar por motivos diferentes. Se mudar a regra de cálculo, a classe muda. Se mudar o banco, a classe muda. Se mudar o serviço de e-mail, a classe também muda.

Código corrigido:

```
class Pedido {  
    constructor(  
        public cliente: string,  
        public valor: number  
    ) {}  
  
    public calcularTotal(): number {  
        return this.valor * 1.1;  
    }  
}  
  
class PedidoRepository {  
    public salvar(pedido: Pedido): void {  
        console.log(`Salvando pedido de ${pedido.cliente}`);  
    }  
}  
  
class EmailService {  
    public enviarConfirmacao(pedido: Pedido): void {  
        console.log(`Enviando e-mail para ${pedido.cliente}`);  
    }  
}
```

Agora cada classe tem uma responsabilidade principal.

Resposta 2 - Factory

Factory é um padrão que **centraliza a criação de objetos**.

Ele resolve o problema de várias chamadas `new ClasseConcreta()` espalhadas pelo código.

Exemplo ruim:

```
const notificacao = new EmailNotificacao();
```

Se isso estiver espalhado em muitos lugares e amanhã o sistema precisar trocar para SMS, será necessário alterar vários pontos.

Com Factory:

```
interface Notificacao {
    enviar(mensagem: string): void;
}

class EmailNotificacao implements Notificacao {
    public enviar(mensagem: string): void {
        console.log(`E-mail: ${mensagem}`);
    }
}

class SmsNotificacao implements Notificacao {
    public enviar(mensagem: string): void {
        console.log(`SMS: ${mensagem}`);
    }
}

class NotificacaoFactory {
    public static criar(tipo: string): Notificacao {
        if (tipo === "email") {
            return new EmailNotificacao();
        }

        if (tipo === "sms") {
            return new SmsNotificacao();
        }

        throw new Error("Tipo inválido");
    }
}
```

Uso:

```
const notificacao = NotificacaoFactory.criar("email");
notificacao.enviar("Olá");
```

A criação fica centralizada, reduzindo acoplamento e facilitando mudanças.

Resposta 3 - Inversão de Dependência

O princípio violado é a **Inversão de Dependência**.

`PedidoService` depende diretamente da classe concreta `EmailService`.

Isso gera alto acoplamento porque, se o envio mudar para SMS, WhatsApp ou Push, será necessário modificar `PedidoService`.

Código corrigido:

```
interface NotificacaoService {
    enviar(mensagem: string): void;
}

class EmailService implements NotificacaoService {
    public enviar(mensagem: string): void {
        console.log(`Enviando e-mail: ${mensagem}`);
    }
}

class SmsService implements NotificacaoService {
    public enviar(mensagem: string): void {
        console.log(`Enviando SMS: ${mensagem}`);
    }
}

class PedidoService {
    constructor(private notificacaoService: NotificacaoService) {}

    public finalizarPedido(): void {
        this.notificacaoService.enviar("Pedido finalizado");
    }
}
```

Uso:

```
const service = new PedidoService(new EmailService());
service.finalizarPedido();
```

Agora `PedidoService` depende de uma abstração.

Resposta 4 - Observer

Diagrama textual:

```

+-----+
|           Subject           |
+-----+
| - observers: Observer[]    |
+-----+
| + addObserver(o: Observer) |
| + removeObserver(o: Observer) |
| + notifyObservers()        |
+-----+
      |
      | mantém lista de
      v
+-----+
|           Observer          |
+-----+
| + update()                  |
+-----+
      ^
      |
+-----+
|       ConcreteObserver      |
+-----+
| + update()                  |
+-----+

+-----+
|       ConcreteSubject       |
+-----+
| - state                     |
+-----+
| + setState()                |
| + getState()                |
+-----+

```

Aplicando ao exemplo:

```

Subject/ConcreteSubject = Temperatura
Observer = Termometro
ConcreteObserver = TermometroDigital, TermometroAnalogico, TermometroWeb

```

Funcionamento:

A classe **Temperatura** mantém uma lista de observadores. Quando o valor da temperatura muda, ela chama **notifyObservers()**. Esse método percorre a lista e chama **update()** em cada termômetro. Assim, **Temperatura** não depende das classes concretas dos termômetros.

Resposta 5 - Aberto/Fechado

O princípio violado é o **Princípio Aberto/Fechado**.

A classe `CalculadoraDesconto` precisa ser modificada sempre que surgir um novo tipo de cliente. Isso acontece porque ela usa vários `if` para decidir o comportamento.

Correção usando Strategy:

```
interface DescontoStrategy {
    calcular(valor: number): number;
}

class DescontoComum implements DescontoStrategy {
    public calcular(valor: number): number {
        return valor * 0.05;
    }
}

class DescontoVip implements DescontoStrategy {
    public calcular(valor: number): number {
        return valor * 0.1;
    }
}

class DescontoPremium implements DescontoStrategy {
    public calcular(valor: number): number {
        return valor * 0.15;
    }
}

class CalculadoraDesconto {
    constructor(private strategy: DescontoStrategy) {}

    public calcular(valor: number): number {
        return this.strategy.calcular(valor);
    }
}
```

Nova extensão:

```
class DescontoBlack implements DescontoStrategy {
    public calcular(valor: number): number {
        return valor * 0.2;
    }
}
```

A classe `CalculadoraDesconto` não precisa ser modificada.

Resposta 6 - Singleton

Singleton é um padrão que garante que uma classe tenha **no máximo uma instância**.

Ele normalmente é implementado com:

```
atributo estático para guardar a instância;  
construtor privado;  
método estático getInstance().
```

Exemplo:

```
class Logger {  
    private static instance: Logger;  
  
    private constructor() {}  
  
    public static getInstance(): Logger {  
        if (!Logger.instance) {  
            Logger.instance = new Logger();  
        }  
  
        return Logger.instance;  
    }  
  
    public log(message: string): void {  
        console.log(message);  
    }  
}
```

O construtor é privado para impedir `new Logger()` fora da classe.

Problemas do Singleton:

```
pode criar acoplamento global;  
pode esconder dependências;  
pode dificultar testes;  
pode virar uma variável global disfarçada;  
em sistemas concorrentes, pode exigir cuidado com múltiplas threads.
```

Resposta 7 - Proxy

Proxy é um objeto intermediário entre o cliente e o objeto real.

No exemplo de busca de livros:

```
Objeto real = BookSearch
Proxy = BookSearchProxy
Cliente = código que pesquisa livros
```

O Proxy pode verificar se o livro está no cache antes de chamar o objeto real.

Exemplo:

```
interface Search {
    getBook(isbn: string): string;
}

class BookSearch implements Search {
    public getBook(isbn: string): string {
        console.log("Buscando na fonte original");
        return "Livro encontrado";
    }
}

class BookSearchProxy implements Search {
    private cache = new Map<string, string>();

    constructor(private realSearch: Search) {}

    public getBook(isbn: string): string {
        const cached = this.cache.get(isbn);

        if (cached) {
            return cached;
        }

        const book = this.realSearch.getBook(isbn);
        this.cache.set(isbn, book);

        return book;
    }
}
```

Outros requisitos não-funcionais com Proxy:

```
logging;
controle de acesso;
validação;
auditoria;
limitação de tentativas.
```

Resposta 8 - Adapter

Adapter é usado para adaptar uma classe com interface incompatível à interface esperada pelo sistema.

No exemplo dos projetores:

```
Target = Projetor
Adapter = AdaptadorProjetorSamsung
Adaptee = ProjetorSamsung
```

Exemplo:

```
interface Projetor {
    ligar(): void;
}

class ProjetorSamsung {
    public turnOn(): void {
        console.log("Samsung ligado");
    }
}

class AdaptadorProjetorSamsung implements Projetor {
    constructor(private samsung: ProjetorSamsung) {}

    public ligar(): void {
        this.samsung.turnOn();
    }
}
```

O Adapter ajuda principalmente na **Inversão de Dependência**, porque o cliente depende da interface **Projetor**, e não da classe concreta **ProjetorSamsung**.

Também pode favorecer o Aberto/Fechado, pois novos projetores podem ser integrados criando novos adaptadores.

Resposta 9 - Facade

Facade fornece uma interface simples para um subsistema complexo.

No exemplo do Home Theater, o subsistema possui:

```
TV;
Projetor;
Receiver;
Player de mídia;
```



```
Sistema de som;  
Luz ambiente.
```

Sem Facade, o cliente precisa ligar e configurar tudo manualmente.

Com Facade:

```
class HomeTheaterFacade {  
    public assistirFilme(): void {  
        console.log("Ligando TV");  
        console.log("Ligando projetor");  
        console.log("Ajustando som");  
        console.log("Reduzindo luz");  
        console.log("Reproduzindo filme");  
    }  
}
```

O cliente chama apenas:

```
homeTheater.assistirFilme();
```

Isso reduz acoplamento e melhora Information Hiding, porque o cliente não conhece os detalhes internos.

Cuidado: se a Facade fizer coisas demais, pode violar **Responsabilidade Única**.

Resposta 10 - Decorator

Decorator adiciona responsabilidades dinamicamente por composição.

Na cafeteria, criar uma classe para cada combinação seria ruim:

```
CafeComLeite;  
CafeComChantilly;  
CafeComLeiteEChantilly;  
CafeComLeiteCanelaChantilly;  
...
```

Isso gera explosão de classes.

Com Decorator:

```
interface Bebida {  
    getDescricao(): string;  
    getPreco(): number;
```

```
}

class CafeExpresso implements Bebida {
    public getDescricao(): string {
        return "Café expresso";
    }

    public getPreco(): number {
        return 5;
    }
}

abstract class BebidaDecorator implements Bebida {
    constructor(protected bebida: Bebida) {}

    public getDescricao(): string {
        return this.bebida.getDescricao();
    }

    public getPreco(): number {
        return this.bebida.getPreco();
    }
}

class Leite extends BebidaDecorator {
    public getDescricao(): string {
        return `${this.bebida.getDescricao()} + leite`;
    }

    public getPreco(): number {
        return this.bebida.getPreco() + 2;
    }
}

class Chantilly extends BebidaDecorator {
    public getDescricao(): string {
        return `${this.bebida.getDescricao()} + chantilly`;
    }

    public getPreco(): number {
        return this.bebida.getPreco() + 3;
    }
}
```

Uso:

```
const bebida: Bebida = new Chantilly(new Leite(new CafeExpresso()));
```

Decorator respeita Aberto/Fechado porque novos adicionais são criados como novas classes, sem alterar a bebida base.

Resposta 11 - Strategy

O padrão adequado é **Strategy**.

O princípio violado é o **Aberto/Fechado**, pois `CalculadoraFrete` precisa ser alterada sempre que surgir um novo tipo de frete.

Código corrigido:

```
interface FreteStrategy {
    calcular(peso: number): number;
}

class FreteSedex implements FreteStrategy {
    public calcular(peso: number): number {
        return peso * 10;
    }
}

class FretePac implements FreteStrategy {
    public calcular(peso: number): number {
        return peso * 5;
    }
}

class FreteTransportadora implements FreteStrategy {
    public calcular(peso: number): number {
        return peso * 8;
    }
}

class CalculadoraFrete {
    constructor(private strategy: FreteStrategy) {}

    public calcular(peso: number): number {
        return this.strategy.calcular(peso);
    }
}
```

Nova extensão:

```
class FreteMotoboy implements FreteStrategy {
    public calcular(peso: number): number {
        return 15;
    }
}
```

Resposta 12 - Template Method

Template Method define o esqueleto de um algoritmo em uma classe abstrata e deixa alguns passos para as subclasses.

No exemplo de salário:

```
abstract class Funcionario {  
    constructor(protected salario: number) {}  
  
    protected abstract calcularPrevidencia(): number;  
    protected abstract calcularPlanoSaude(): number;  
  
    public calcularSalarioLiquido(): number {  
        return this.salario - this.calcularPrevidencia() - this.calcularPlanoSaude();  
    }  
}
```

Subclasse:

```
class FuncionarioCLT extends Funcionario {  
    protected calcularPrevidencia(): number {  
        return this.salario * 0.08;  
    }  
  
    protected calcularPlanoSaude(): number {  
        return 200;  
    }  
}
```

O método template é `calcularSalarioLiquido()`.

Ele se relaciona com Inversão de Controle porque a classe pai controla o fluxo e chama métodos implementados pelas subclasses.

Resposta 13 - Visitor

Visitor permite adicionar operações a uma hierarquia de classes sem colocar essas operações diretamente nas classes.

Exemplo:

```
interface Veiculo {  
    accept(visitor: VeiculoVisitor): void;  
}  
  
interface VeiculoVisitor {
```

```
visitCarro(carro: Carro): void;  
visitOnibus(onibus: Onibus): void;  
visitMotocicleta(motocicleta: Motocicleta): void;  
}
```

O método `accept(visitor)` permite que o objeto visitado chame o método correto do visitor.

Exemplo:

```
class Carro implements Veiculo {  
    public accept(visitor: VeiculoVisitor): void {  
        visitor.visitCarro(this);  
    }  
}
```

Vantagem:

facilita adicionar novas operações, como imprimir, salvar ou calcular imposto.

Desvantagem:

se adicionar uma nova classe na hierarquia, como Caminhao, todos os Visitors precisam ser alterados.

Também pode quebrar encapsulamento.

Resposta 14 - Inversão de Controle

Inversão de Controle ocorre quando o fluxo principal é controlado por uma estrutura externa, e não diretamente pelo código do usuário.

No Template Method, a classe abstrata controla a ordem dos passos e chama métodos das subclasses.

Exemplo:

```
abstract class Processo {  
    public executar(): void {  
        this.iniciar();  
        this.processar();  
        this.finalizar();  
    }  
  
    protected iniciar(): void {  
        console.log("Iniciando");  
    }  
}
```

```
}

protected abstract processar(): void;

protected finalizar(): void {
    console.log("Finalizando");
}
}
```

Em frameworks, o framework chama o código do usuário em momentos específicos.

Diferença:

Inversão de Controle: fala sobre quem controla o fluxo.
Inversão de Dependência: fala sobre depender de abstrações, não de classes concretas.

Resposta 15 - Framework vs Biblioteca

A principal diferença é quem controla o fluxo.

Biblioteca: você chama o código dela.
Framework: ele chama o seu código.

Biblioteca:

```
const resultado = Math.max(10, 20);
```

Meu código chama a função.

Framework:

```
function App() {
    return <h1>Olá</h1>;
}
```

O React chama o componente quando precisa renderizar.

Em uma biblioteca, o controle principal fica com o programador. Em um framework, a estrutura principal controla parte do fluxo e chama o código do programador.

Resposta 16 - Design for Change

Design for Change significa projetar o software pensando em mudanças futuras.

Padrões de projeto ajudam porque criam pontos de extensão.

Exemplo:

```
Strategy permite adicionar novos algoritmos.  
Observer permite adicionar novos observadores.  
Decorator permite adicionar novos adicionais.  
Adapter permite integrar novas APIs.
```

Relação com Aberto/Fechado:

```
novas funcionalidades são adicionadas por extensão, sem modificar classes  
existentes.
```

Relação com baixo acoplamento:

```
classes dependem de abstrações e não de detalhes concretos.
```

Exemplo:

```
interface Notificacao {  
    enviar(mensagem: string): void;  
}  
  
class EmailNotificacao implements Notificacao {  
    public enviar(mensagem: string): void {  
        console.log(mensagem);  
    }  
}
```

Se surgir SMS, cria-se uma nova classe, sem alterar quem usa `Notificacao`.

Resposta 17 - Overengineering

Overengineering ocorre quando a solução fica mais complexa do que o problema exige.

Padrões podem virar overengineering quando usados sem necessidade real de mudança.

Exemplo exagerado:

```
interface Saudacao {  
    executar(): void;  
}  
  
class SaudacaoPadrao implements Saudacao {  
    public executar(): void {  
        console.log("Olá");  
    }  
}  
  
class SaudacaoFactory {  
    public static criar(): Saudacao {  
        return new SaudacaoPadrao();  
    }  
}
```

Se o objetivo era apenas imprimir "Olá", bastaria:

```
class Saudacao {  
    public executar(): void {  
        console.log("Olá");  
    }  
}
```

Mais classes e interfaces nem sempre significam melhor projeto. Se a mudança é improvável, a solução simples pode ser melhor.

Resposta 18 - Segregação de Interfaces

O princípio violado é a **Segregação de Interfaces**.

A interface **Trabalhador** obriga **Robo** a implementar métodos que não fazem sentido:

```
comer();  
dormir();
```

Código corrigido:

```
interface Trabalhavel {  
    trabalhar(): void;  
}  
  
interface Alimentavel {  
    comer(): void;  
}
```



```
interface Descansavel {
    dormir(): void;
}

class Robo implements Trabalhavel {
    public trabalhar(): void {
        console.log("Robô trabalhando");
    }
}

class Humano implements Trabalhavel, Alimentavel, Descansavel {
    public trabalhar(): void {
        console.log("Humano trabalhando");
    }

    public comer(): void {
        console.log("Humano comendo");
    }

    public dormir(): void {
        console.log("Humano dormindo");
    }
}
```

Agora cada classe implementa apenas o que usa.

Resposta 19 - Liskov

O princípio violado é a **Substituição de Liskov**.

Pinguim não substitui corretamente **Ave** porque **Ave** promete o comportamento **voar()**, mas **Pinguim** não consegue cumprir esse comportamento.

Código corrigido:

```
class Ave {
    public comer(): void {
        console.log("Comendo");
    }
}

class AveVoadora extends Ave {
    public voar(): void {
        console.log("Voando");
    }
}

class Pardal extends AveVoadora {}

class Pinguim extends Ave {
```

```
public nadar(): void {  
    console.log("Nadando");  
}  
}
```

Agora somente aves que realmente voam herdam de [AveVoadora](#).

Resposta 20 - Questão integrada

A classe [RelatorioService](#) pode estar violando vários princípios.

Possíveis violações:

Responsabilidade Única:

porque busca dados, gera PDF, exporta CSV e envia e-mail.

Aberto/Fechado:

porque decide o formato com if e precisará mudar para novos formatos.

Inversão de Dependência:

porque instancia diretamente EmailService e MySQLRepository.

Segregação/coesão:

dependendo da estrutura, pode misturar responsabilidades demais.

Padrões que poderiam ajudar:

Strategy:

para formatos de exportação diferentes.

Factory:

para centralizar criação de exportadores.

Proxy:

para log, cache ou validação.

Facade:

se houver um subsistema complexo de relatórios.

Adapter:

se houver integração com uma API externa.

Observer:

se vários interessados precisarem ser notificados quando o relatório for gerado.

Refatoração geral:

RelatorioService deve coordenar o processo.
RelatorioRepository deve buscar dados.
ExportadorRelatorio deve ser uma interface.
ExportadorPDF e ExportadorCSV implementam a interface.
EmailService deve implementar uma interface NotificacaoService.
RelatorioService deve receber dependências por construtor.

Exemplo simplificado:

```
interface RelatorioRepository {  
    buscarDados(): string[];  
}  
  
interface ExportadorRelatorio {  
    exportar(dados: string[]): void;  
}  
  
interface NotificacaoService {  
    enviar(mensagem: string): void;  
}  
  
class RelatorioService {  
    constructor(  
        private repository: RelatorioRepository,  
        private exportador: ExportadorRelatorio,  
        private notificacao: NotificacaoService  
    ) {}  
  
    public gerar(): void {  
        const dados = this.repository.buscarDados();  
        this.exportador.exportar(dados);  
        this.notificacao.enviar("Relatório gerado");  
    }  
}
```

Essa solução separa responsabilidades, reduz acoplamento e permite novos formatos por extensão.